

Recitation 1:

Cache Side-Channel Attacks

2026-09-09



Today's agenda

- Lab 1 overview
- Intro to the lab machine
- Set-associative cache review
- Measuring memory access latency

Lab 1



High-level overview

- Four parts
 - Flush+Reload (20%)
 - Prime+Probe (30%)
 - Decrypt the SPECK cipher (30%)
 - Build a covert channel (20%)
- The lab materials and your SSH key to the lab machine are available on Canvas

This lab is challenging! Please start early!



Part 1: Flush+Reload (20%)

Victim code:

```
1  // Allocate a large memory buffer
2  char *buf = get_buffer();
3
4  // Create a random flag in [0, 1024)
5  int flag = random(0, 1024);
6  printf(flag);
7
8  // Main Loop
9  while (true) {
10     value = load(buf + flag * 128);
11 }
```

- Goal: decode the flag from the cache side channel
- The buffer `buf` is a *memory-mapped file*, **shared** between the attacker and the victim at the same physical address



Part 2: Prime+Probe (30%)

Victim code:

```
1  // Compute a random offset and the flag
2  char *buf = get_buffer();
3  int offset = rand() % 0x10000;
4  buf += offset;
5  int flag = offset >> 6;
6  printf(flag);
7
8  while (true) {
9      for (int i = 0; i < 4; i++) {
10         (*(char *)(buf + i * 0x10000))++;
11     }
12 }
```

- Goal: decode the flag
- The buffer `buf` is a **NOT** shared
- Notice the constants here:
 - `0x10000`
 - `6` (in `flag = offset >> 6`)
- Which level of cache should you target?



Part 3: Decrypt the SPECK cipher (30%)

```
1  // Generate a random plaintext and pick a random encryption function
2  block pt = random_plaintext();
3  fn encrypt = speck_encrypt_functions[rand() % 64];
4
5  // Encrypt the plaintext 1M times to make it *extra* secure!
6  block ref = pt;
7  for (long i = 0; i < 1000000; i++) ref = encrypt(ref);
8  printf(ref);
9
10 // Double check the encryption
11 while(1) {
12     block check = pt;
13     for (long i = 0; i < 1000000; i++) check = encrypt(check);
14     if (check != ref) printf("ERROR!");
15 }
```



The SPECK block cipher

```
1 void speck_encrypt(uint16_t *x, uint16_t *y)
2 {
3     for (unsigned i = 0; i < SPECK_ROUNDS; i++) {
4         *x = ROR16(*x, 7);
5         *x = *x + *y;
6         *x = *x ^ round_key[i];
7
8         *y = ROL16(*y, 2);
9         *y = *y ^ *x;
10    }
11 }
```

- There is no memory loads (`round_key[i]` is a constant hardcoded in victim's code)
- What can we exploit?



The signals in the cache side channel

When the victim picks `encrypt = speck_g8_r0` and calls `encrypt(pt)`, it will execute the following functions.

- `speck_g8_r0` (at `0x02200`)
- `speck_g8_r1` (at `0x03200`)
- `speck_g8_r2` (at `0x04200`)
- ...
- `speck_g8_r21` (at `0x17200`)

When the CPU executes these functions, it will load the instructions from the memory, thus it will create memory traffic and leave signals in the cache side channel.

Part 4: Dead drop – an evil chat client (20%)

You need to implement a covert channel messaging app (with whatever side channel you like).



```
joey@coffeelakerf: ~/deaddr...  
joey@coffeelakerf:~/deaddrop/build/bin$ ./sender  
Please type a message (exit to stop).  
< Hello, world!  
Bitrate: 47337.28 Bytes/second  
< Hi!  
Bitrate: 47337.28 Bytes/second  
< |  
joey@coffeelakerf:~/deaddrop/build/bin$ ./receiver  
Listening...  
> Hello, world!  
> Hi!  
[0] 0:./sender* "coffeelakerf" 17:10 08-Sep-26
```



The lab machine



The CPU of the lab machine

- CPU: Intel i9-9900
- 8 physical cores (16 logical cores with SMT)
 - Use `lscpu -e=CPU,CORE` to verify this
- You will be assigned to one physical core when you build and run your code
 - A core will be allocated for you for 30 minutes
 - You will enter a queue when all cores are occupied
 - Run `cpu-client` for more information



Microarchitecture of the lab machine

- Microarchitecture: Coffee Lake Refresh
- Let's use `lscpu` and `getconf -a | grep CACHE` to fill in the following table:

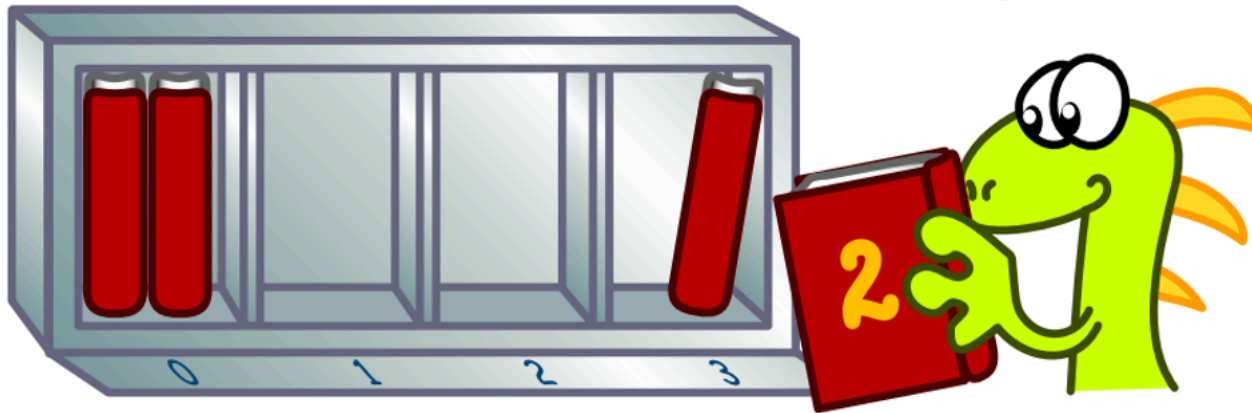
Cache	Cache Line Size	Size	Number of Ways (Associativity)	Number of Sets
L1-Data	64	32 KB (*8)	8	64
L1-Instruction	64	32 KB (*8)	8	64
L2	64	256 KB (*8)	4	1024
L3	64	16 MB	16	16 * 1024

$\text{\#Sets} = \text{Size} / (\text{Line size} \times \text{\#Ways})$

Set-associative cache review

Set-associative cache

2-Way Set Associative



Tag	Index	Offset
-----	-------	--------

This cache is made up of sets that can fit two blocks each. The index is now used to find the set, and the tag helps find the block within the set.

4-Way Set Associative



Tag	Index	Offset
-----	-------	--------

Each set here fits four blocks, so there are fewer sets. As such, fewer index bits are needed.



Exercise: mappings of the SPECK round functions

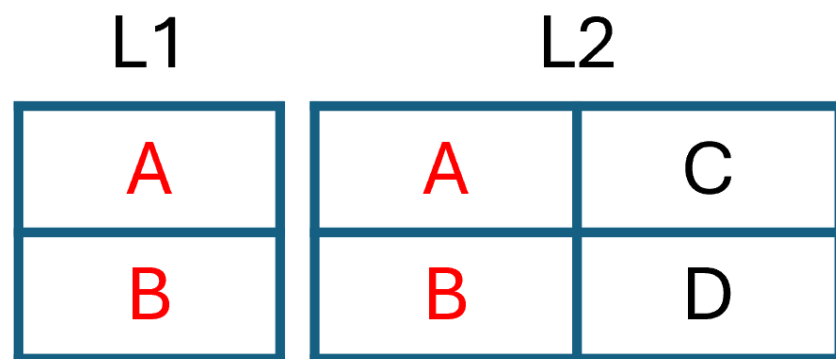
- The addresses of the round functions:
 - Group 1: 0x2040, 0x3040, 0x4040, ...
 - Group 8: 0x2200, 0x3200, 0x4200, ...
- The lab machine has 64 sets for the L1
- The cache line size is 64 bytes

Address	0x2040	0x3040	0x4040	0x2200	0x3200	0x4200
Set	1	1	1	8	8	8
Tag	2	3	4	2	3	4

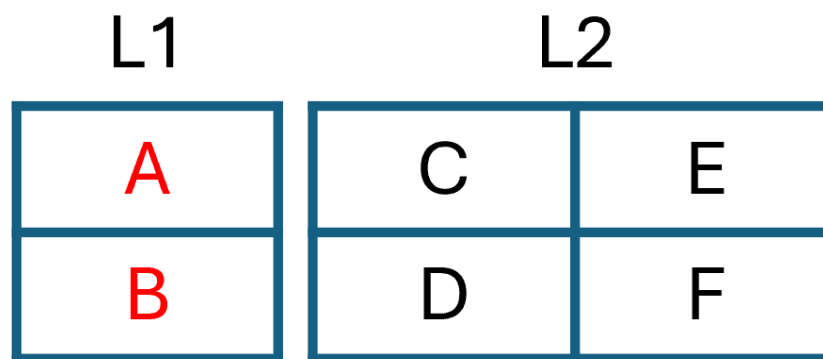
Cache inclusion policy

Assume a 2-level cache:

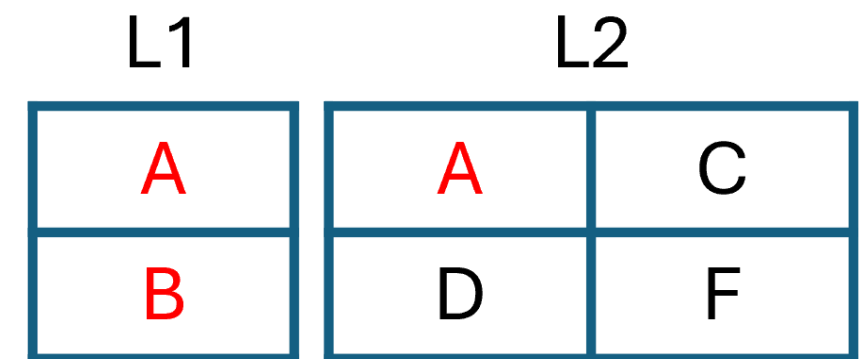
- Inclusive: lines in L1 exist in L2
- Exclusive: lines in L1 do not exist in L2
- Non-inclusive: lines in L1 may or may not exist in L2



▲ Inclusive



▲ Exclusive

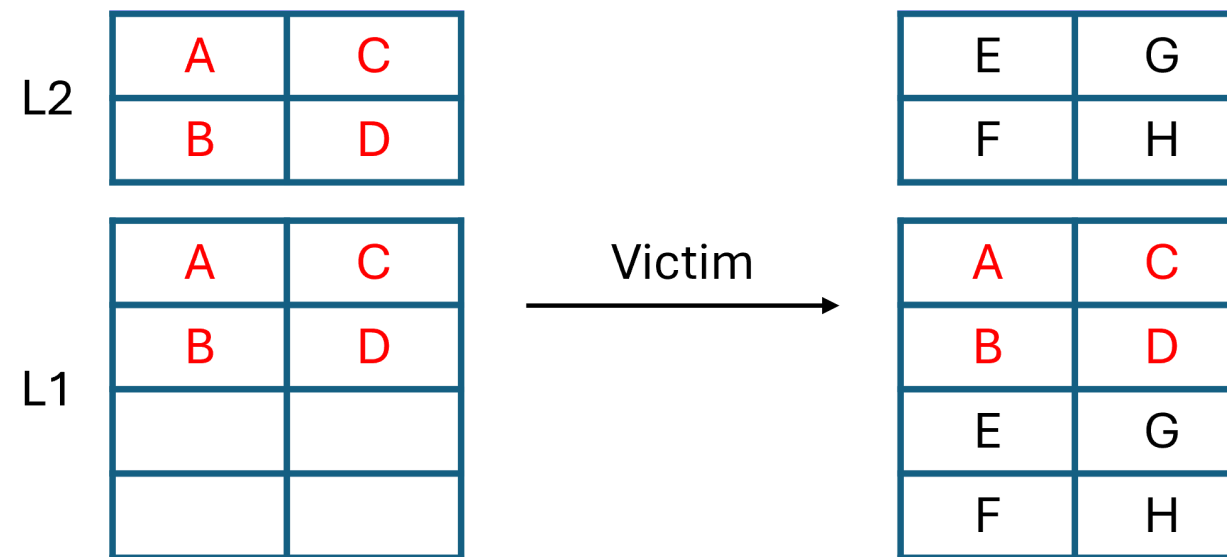


▲ Non-Inclusive

Prime+Probe on non-inclusive cache

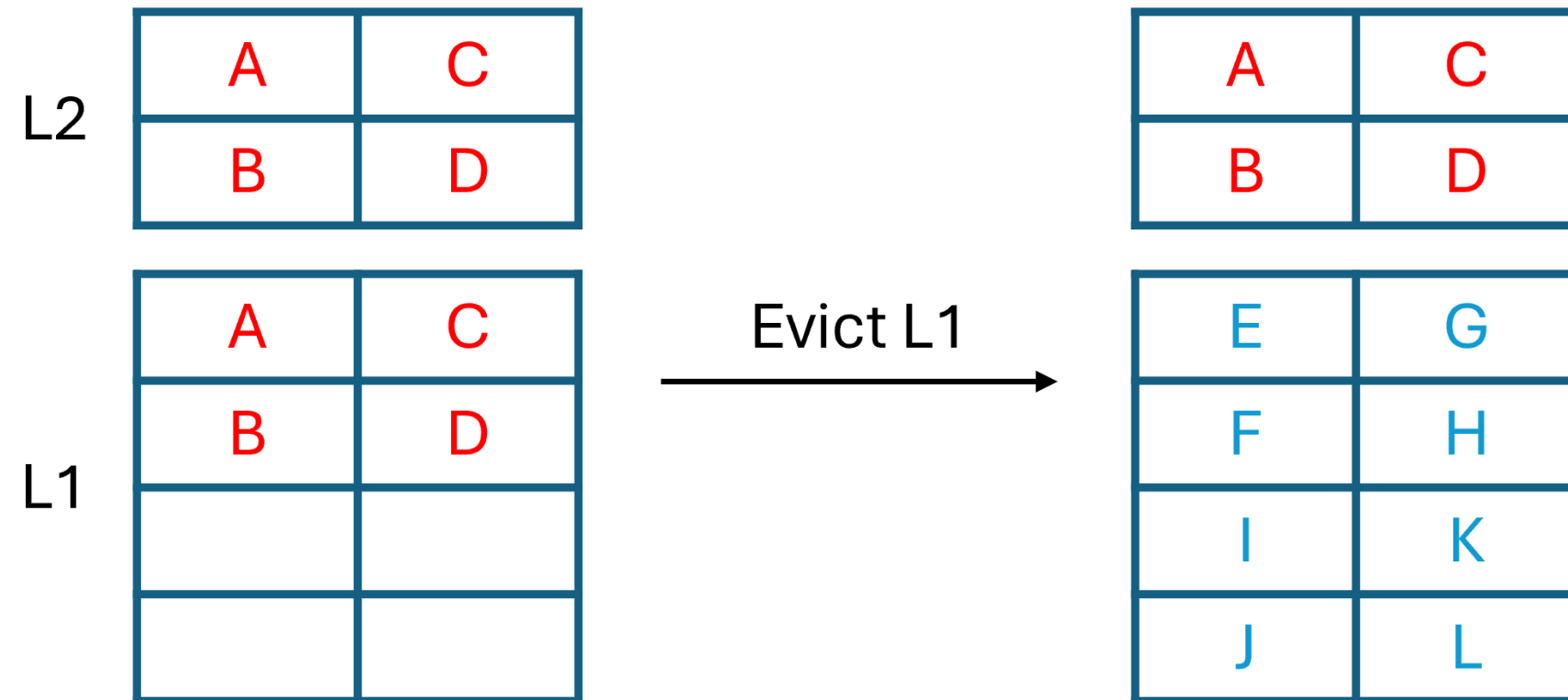
When you Prime+Probe a non-inclusive L2 (e.g., our lab machine):

1. Attacker primes lines A, B, C, and D
2. The victim accesses lines E, F, G, and H
3. What's the access latency when the attacker probes lines A, B, C, and D?



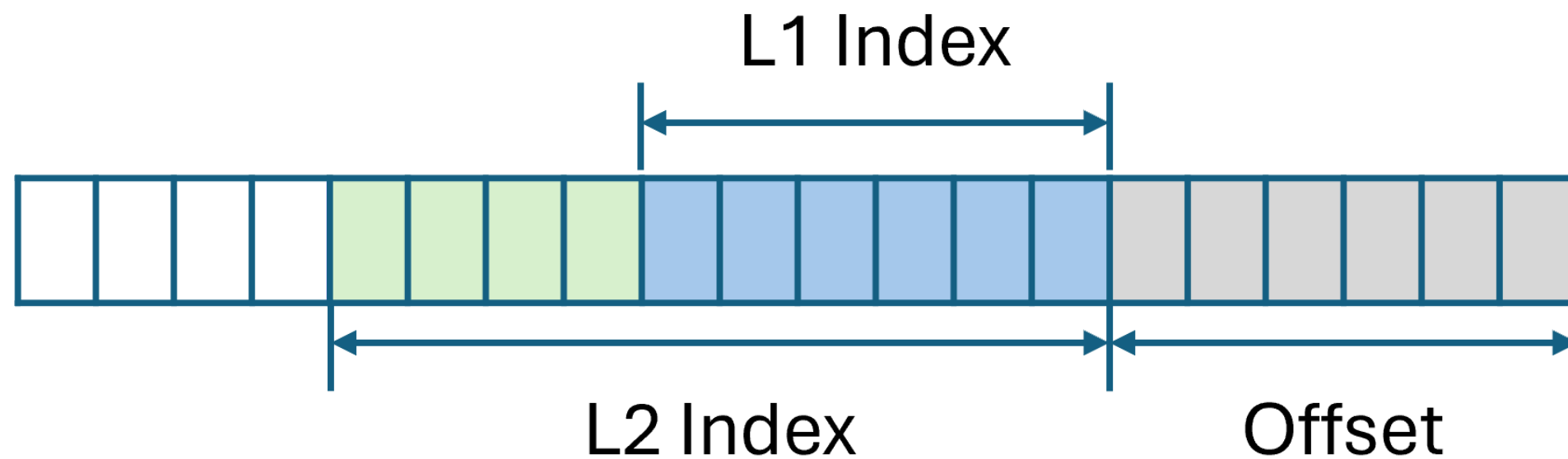
How to Prime+Probe correctly?

- Idea: evict the attacker lines in the L1 after prime
- How to find lines E ~ L?



Lines in the same L1 set but in different L2 sets

- Suppose the attacker primes and probes set x in the L2
- The attacker's lines will map to set $c = x \bmod 64$ in the L1
- Lines in set $y = x + 64 \cdot k$ in the L2 will map to the same set in the L1



Measuring memory access latency



Exercise: measuring cache hit & DRAM latency

Let's measure lab machine's L1, L2, L3, and DRAM access latency.

```
1  // Measure L1 access latency, store results in l1_latency array
2  for (int i = 0; i < SAMPLES; i++) {
3      // TODO: bring the target cache line into L1
4
5      l1_latency[i] = measure_one_block_access_time((uint64_t)target_buffer);
6  }
```

Some pitfalls:

- Do **NOT** call `printf`, `sleep`, or other library functions or system calls in the timing loop
- Beware of the [zero page](#). Always write to a freshly allocated buffer.



Exercise: measuring the cache line size

- Cache lines may not always be 64 bytes in size
- Design an experiment to determine the size of a cache line